Data Engine Merge: Decision Log

This document records every design decision made while merging the repository's branches into one complete data engine on `main`. Each entry says what changed, what the old design was, why it did not work, what replaced it, and why the new design is better. It is updated as the merge proceeds.

1. Where things stand

Branch	Relationship to the others	Treatment
`main`	Ten commits, identical to `Moving_backend_CRUDS`.	Receives `dev` last.
`Moving_backend_CRUDS`	Same commit as `main`.	Nothing to merge.
`ccr-9bc99585-jjmdov`	Already contained in `dev`.	Nothing to merge.
`crud-for-ai-engine`	Already contained in `dev`.	Nothing to merge.
`dev`	33 commits ahead of `main`: atoms, memory, files, meetings, planning, collaboration, Row Level Security (RLS) and JWT scopes.	Base of the merge.
`feature/schema-and-basic-cruds`	Diverges from `dev`: 19 commits of its own, with a root-level layout (`db/`, `domain/`, `schemas/`), synchronous SQLAlchemy, Alembic and an Organization entity.	Selected pieces ported into `dev`.

Only one branch truly diverges, so the work is one careful port rather than a four-way merge.

2. Decisions

Each entry uses the same fields: **Change**, **Old design**, **Why it did not work**, **New design**, **Why it is better**.

D1. `dev` is the base; the feature branch is ported piece by piece

- **Change:** the final `main` is built from `dev`. Parts of `feature/schema-and-basic-cruds` are re-implemented inside `dev`'s `app/` layout instead of running a git merge.
- **Old design:** two incompatible code bases. `feature` puts packages at the repository root and uses synchronous sessions. `dev` uses an `app/` package, workspace-scoped services and RLS.
- **Why it did not work:** both branches define `users`, `workspaces`, `members`, `teams`, `projects`, `sprints`, `work_items` and `tasks` with different columns, types and rules. A wholesale merge conflicts in almost every file and would leave duplicate models for the same tables.
- **New design:** one code base under `app/`, with the feature branch's missing capabilities added to it.
- **Why it is better:** `dev` already carries the larger and more security-critical part of the system (atoms, memory, files, meetings, RLS, tokens). Re-porting that onto the feature layout would risk regressions in the parts that matter most.

D2. The Organization entity is removed

- **Change:** the `organizations` table, every `organization_id` column, the `X-Organization-Id` header and the `/organizations/{orgId}` routes are not carried over.
- **Old design:** the feature branch put an Organization above Workspace, and stamped `organization_id` on projects, tasks, work items, sprints, members and teams. Callers selected the organization with a request header.
- **Why it did not work:** `dev` already isolates tenants by `workspace_id`, enforced by composite foreign keys and RLS. A second tenant level would mean two isolation keys on every row, two checks on every query, and a caller-supplied header deciding the scope. A header can be changed by the client; a signed token cannot.
- **New design:** Workspace is the single tenant boundary. The workspace comes from the signed token and never from a header or request argument.
- **Why it is better:** one key to isolate on, one place it is checked, and no way for a caller to select a different tenant by editing a request. This also matches the requirement that all access is determined by who the token says the user is.
- **Revisit if:** a customer needs several workspaces under one billing or administration umbrella. That can be added later as a parent of workspaces without changing the row-level isolation key.

D3. One JWT identifies the user, the workspace and the permissions

- **Change:** a single token shape carries `sub` (user id), `workspace_id`, `mem` (member id) and `scopes`. A new `admin` scope marks workspace administrators. Humans and atoms use the same shape; atoms add their `context` (personal or workspace).
- **Old design:** `dev` tokens identified the workspace and optionally the member, and were aimed at agents. The feature branch used a shared service key plus an `X-Organization-Id` header and had no concept of which person was calling.
- **Why it did not work:** a service key plus a header proves that the caller is the trusted backend, not who the person is, so nothing could be attributed to a user and no per-person rule could be enforced.
- **New design:** every request carries a token that says who the person is, which workspace they act in, and what they may do. The service key is kept only for the backend to mint tokens.
- **Why it is better:** authorization, audit (who did this) and tenant isolation all come from one signed source. Adding a new rule means adding a scope, not a new header or code path.

D4. Permissions are scopes in the token plus a live role check

- **Change:** the token's scope gates each route. In addition, each request looks up the member in the database and confirms they are still active and hold the required role.
- **Old design:** scopes alone, fixed when the token is issued.
- **Why it did not work:** a scope-only token stays valid until it expires. A member who is removed or demoted would keep their access for that time.
- **New design:** scopes are the coarse gate; the live lookup is the source of truth for membership and role.
- **Why it is better:** removal and demotion take effect immediately. The cost is one small indexed lookup per request.

D5. Port sprints and users routes; do not port the rest of the feature CRUD

- **Change:** the sprint and user routes from the feature branch are re-implemented in `dev`'s planning and identity services. The feature branch's project, task, work item, member and team routes are not ported.
- **Old design:** the feature branch has a full second CRUD layer for every planning and identity resource.
- **Why it did not work:** `dev` already provides projects, work items, tasks, members, teams and workspaces with workspace scoping and tests. Porting the duplicates would create two routes for the same resource. `dev` has a `Sprint` table but no sprint routes, and a `User` table but no user routes.
- **New design:** fill the two real gaps and keep `dev`'s implementation for everything else.
- **Why it is better:** less code, one implementation per resource, and no behavioural differences to reconcile.
- **Not ported:** the `project_teams` join table. `dev` keeps a single nullable `projects.team_id`. Revisit if a project must belong to several teams.

D6. `dev`'s lowercase string values win over the feature branch's uppercase enums

- **Change:** statuses, priorities and types stay as lowercase strings (for example `backlog`, `none`).
- **Old design:** the feature branch used native Postgres ENUM types with uppercase values (`TODO`, `IN_PROGRESS`, `LOW` to `URGENT`).
- **Why it did not work:** the AI engine, atoms and memory tools already read and write the lowercase values. Switching vocabulary would break them and require a data migration. Native ENUM types are also harder to change: adding or removing a value requires a schema change.
- **New design:** one vocabulary, `dev`'s, applied across the API.
- **Why it is better:** nothing that works today has to change, and adding a value does not require a type migration.

D7. The API keeps `dev`'s snake_case and error format

- **Change:** the feature branch's camelCase JSON and its `DomainError` code envelope (for example `ORGANIZATION_ID_REQUIRED`) are not adopted.
- **Old design:** camelCase requests and responses with a custom error envelope.
- **Why it did not work:** the AI engine consumes `dev`'s snake_case contract and error shapes. Changing them would break an existing client for no functional gain.
- **New design:** a single contract across the whole engine.
- **Why it is better:** one style to document and test. camelCase can be added later at the edge if the frontend asks for it.

D8. Alembic is adopted with a single fresh baseline

- **Change:** Alembic is used for schema management. Its history starts with one new migration generated from the merged models, not the feature branch's initial migration.
- **Old design:** `dev` creates the schema from the models and an install script, with RLS applied separately. The feature branch had Alembic, but its migration describes tables that no longer exist after this merge.
- **Why it did not work:** the feature branch's migration targets the Organization-based schema, which this merge removes. `dev`'s install script has no history, so a production database could not be upgraded in place.
- **New design:** a baseline migration for the merged schema. Because autogenerate does not emit them, the baseline is edited by hand to include the pgvector extension, RLS policies, triggers (such as task numbering)

and the composite foreign keys.

- **Why it is better:** every later schema change is versioned and reviewable, and production can be upgraded safely.
- **Risk to check:** the baseline must produce the same schema as the install script. We verify this by building a database both ways and comparing them. `dev` already has a drift test that uses Alembic's `compare_metadata`, which can be reused for this.
- **Related:** migrations run with `DATABASE_ADMIN_URL` (see D18), and `app/db/install.py` either calls Alembic or is retired once the baseline is trusted.

D9. The feature branch's tests and Postman collection are kept, rewritten

- **Change:** the integration and unit tests and the Postman collection are adapted to the merged system.
- **Old design:** tests and requests written against the Organization header and the uppercase enums.
- **Why it did not work as is:** they would fail against the new auth, vocabulary and layout.
- **New design:** the same coverage, running against `dev`'s harness and JWT auth, with Postman using a Bearer token and `workspace_id`.
- **Why it is better:** we keep the coverage the feature branch built, and the tests exercise the real access rules.

D10. Only one branch needs porting; `dev` then goes into `main` as a fast-forward

- **Change:** `Moving_backend_CRUDS`, `ccr-9bc99585-jjmdov` and `crud-for-ai-engine` are not merged. After the port, `dev` is merged into `main` with no merge commit.
- **Old design:** five remote branches that looked like five things to reconcile.
- **Why it did not work:** it was never five. `Moving_backend_CRUDS` is the same commit as `main`, and the other two are ancestors of `dev`, so `dev` already holds their work.
- **New design:** one port (the feature branch), then one fast-forward of `main`.
- **Why it is better:** nothing can be lost or double-applied, and `main` history stays linear.

D11. The synchronous database session is not carried over

- **Change:** the feature branch's `db/session.py` (`get_engine`, `SessionLocal`, `get_session`) is dropped.
- **Old design:** each request opens a plain SQLAlchemy `Session` through a FastAPI dependency.
- **Why it did not work:** `dev` already has its own data layer that sets the workspace for RLS on each connection. A second session factory would bypass it, and queries from the ported routes would run without the tenant context.
- **New design:** every ported route uses `dev`'s data layer and the workspace from the token.
- **Why it is better:** there is exactly one way to open a tenant-scoped connection, so a new route cannot forget it.

D12. The feature branch's duplicate models and CRUD are not carried over

- **Change:** its models, schemas and CRUD for projects, tasks, work items, members and teams are dropped, along with `project_teams`' CRUD.
- **Old design:** a second implementation of resources `dev` already has, written against `organization_id`.
- **Why it did not work:** two models for one table cannot both be right, and these depend on Organization (see D2).
- **New design:** `dev`'s models and services remain the only implementation.

- **Why it is better:** less code to maintain, and one set of rules per resource.

D13. Only the backend mints tokens, using the service key

- **Change:** the service key is used to mint JWTs and nothing else. It is not accepted on the CRUD routes.
- **Old design:** the feature branch accepted the service key as the credential for CRUD calls; `dev` used it for admin routes.
- **Why it did not work:** a key that can call every route as anyone has no identity, so audit and per-user rules cannot work, and a leak exposes everything.
- **New design:** the backend authenticates the person, then mints a short-lived token for that person and workspace. Routes accept tokens only.
- **Why it is better:** every action is tied to a user and workspace, and the powerful key is used in one place.
- **Revisit if:** a trusted internal service needs to act without a user. It would get its own token type, not the shared key.

D14. `project_teams` is skipped

- **Change:** the many-to-many link between projects and teams is not ported.
- **Old design:** the feature branch lets a project belong to many teams.
- **Why it did not work for now:** `dev` models a project with one optional team (`projects.team_id`, null for workspace-wide), and its planning code and tests depend on that.
- **New design:** `projects.team_id` stays.
- **Why it is better:** no schema or service change for a capability nobody has asked for yet.
- **Revisit if:** a project must be owned by several teams. Add the join table then.

D15. The feature branch's error handling and router wiring are not carried over

- **Change:** `register_error_handlers`, `domain_router` and the `DomainError` classes are dropped from the app setup.
- **Old design:** a custom error envelope with its own codes.
- **Why it did not work:** it follows from D7. `dev`'s API already has its error format, and two formats in one API are harder for clients to handle.
- **New design:** ported routes raise `dev`'s errors.
- **Why it is better:** clients handle one error shape.

D16. The `mcp` version pin needs no change

- **Change:** none. `dev` already pins `mcp>=1.12,<2`.
- **Old design:** the feature branch added the same pin because `mcp` 2.x breaks the fastmcp import.
- **Why it matters:** both branches reached the same fix independently, so there is nothing to reconcile.
- **Why it is better:** one less conflict.

D17. Local Postgres uses `dev`'s pgvector image

- **Change:** the feature branch's `postgres:16-alpine` compose service is dropped. `dev`'s `pgvector/pgvector:pg17` service stays.

- **Old design:** a plain Postgres image.
- **Why it did not work:** the plain image has no `vector` extension, so memory, atoms and skills (which store embeddings) cannot start on it.
- **New design:** `dev`'s compose file, unchanged.
- **Why it is better:** the local database matches what the code needs.

D18. `DATABASE_URL` already exists in `dev`; no second setting is added

- **Change:** none for the application URL. The feature branch's `DATABASE_URL` addition is dropped.
- **Old design:** the feature branch added the setting to `config.py` and `.env.example`.
- **Why it did not work:** `dev` already has `DATABASE_URL` (the connection the API uses, which must not be a superuser or BYPASSRLS role) and `DATABASE_ADMIN_URL` (used only for schema installation).
- **New design:** Alembic migrations run with `DATABASE_ADMIN_URL`. The API keeps using `DATABASE_URL`.
- **Why it is better:** schema changes need elevated rights, and request handling must not have them. Using one URL for both would defeat RLS.

D19. Dependencies: only Alembic changes

- **Change:** `sqlalchemy`, `psycopg`, `pgvector` and the `mcp` pin are already in `dev`. `email-validator` is added only if a ported schema uses `EmailStr`. `alembic` moves from the development requirements to the main requirements, so migrations can run from the container.
- **Old design:** `dev` lists `alembic` as a development-only dependency used by a schema drift test.
- **Why it did not work:** with D8, migrations are part of deployment, not only of tests.
- **New design:** `alembic` is a normal dependency, and the drift test stays.
- **Why it is better:** the deployed image can upgrade its own database.

D20. The Dockerfile copies the migration files

- **Change:** the image also copies `migrations/` and `alembic.ini`. The feature branch's `COPY db`, `COPY schemas` lines are not used.
- **Old design:** the feature branch copied its root-level packages. `dev` copies only `app/`.
- **Why it did not work:** those root-level directories do not exist after the port, and without the migration files the image cannot run `alembic upgrade`.
- **New design:** `COPY app`, `COPY migrations`, `COPY alembic.ini`.
- **Why it is better:** the image contains exactly what it needs to start and to migrate.

D21. Tests run with pytest; `run.sh` gets a `migrate` command

- **Change:** `./run.sh test` runs `pytest`, and `./run.sh migrate` runs `alembic upgrade head`. `pytest` is added to the development requirements.
- **Old design:** `dev` runs `unittest discover`; the feature branch's tests are written for pytest with fixtures.
- **Why it did not work:** pytest fixtures cannot run under `unittest discover`, so the ported tests would be skipped or fail.
- **New design:** pytest runs both styles, because it collects `unittest.TestCase` classes too.
- **Why it is better:** one command runs the old and the ported tests together.

- **To verify:** run the whole suite under pytest before switching the command. If any existing `dev` test behaves differently, keep it under `unittest` until fixed.

D22. Test setup and development requirements

- **Change:** `tests/harness.py` keeps `dev`'s database setup; the feature branch's default test `DATABASE_URL` is not added. `requirements-dev.txt` gains `pytest`.
- **Old design:** the feature branch hard-codes a test URL.
- **Why it did not work:** `dev` already reads `TEST_DATABASE_URL` and has helper code in `tests/db_support.py` that creates the schema and the RLS role, which the hard-coded URL would skip.
- **New design:** ported tests use `dev`'s helpers.
- **Why it is better:** tests run under the same RLS rules as production.

D23. README is merged by hand

- **Change:** the feature branch's README edits are folded into `dev`'s README. Sections on Organization, `X-Organization-Id` and the old test command are not copied.
- **Old design:** README describing the feature branch's setup.
- **Why it did not work:** it describes a layout and an auth model that are gone.
- **New design:** one README that describes the merged system: token auth, `./run.sh postgres`, `./run.sh migrate`, `./run.sh test`.
- **Why it is better:** a new developer can follow it without hitting removed features.

D24. `dev`'s memory documentation is kept

- **Change:** `docs/MEMORY_API.md` and `docs/MEMORY_TOOLS.md` stay.
- **Old design:** the feature branch deleted both files.
- **Why it did not work:** the memory API and tools still exist in `dev` and are used. Deleting the docs would leave them undocumented.
- **New design:** keep the files and update them when memory behaviour changes.
- **Why it is better:** documentation matches the code.

D25. `dev`'s indexes are kept

- **Change:** the feature branch's removal of performance-only indexes is not applied.
- **Old design:** indexes removed "for MVP".
- **Why it did not work:** that removal covered the feature branch's small schema. `dev` has full-text (GIN) and vector indexes that search and recall depend on.
- **New design:** keep every index `dev` has. Indexes on the ported tables follow the same rule: only those a real query needs.
- **Why it is better:** search stays fast, and nothing is dropped without measuring.

D26. Postman uses a Bearer token

- **Change:** the collection and environment drop the `X-Organization-Id` header and the service key. They gain a `token` variable that is sent as `Authorization: Bearer {{token}}`.

- **Old design:** every request sent the header and the service key.
- **Why it did not work:** the server no longer reads them (D2, D13).
- **New design:** a pre-request step or a first request obtains the token, and all other requests reuse it.
- **Why it is better:** the collection exercises the same auth path as the real clients.

D27. Sprint routes added to `dev`'s planning API; audit log accepts dates and UUIDs

- **Change:** `POST/GET /workspaces/{id}/projects/{project_id}/sprints`, and `GET/PATCH/DELETE .../sprints/{sprint_id}`. The list route filters by `status`. Deleting a sprint is a hard delete, and its tasks go back to the backlog.
- **Old design:** the feature branch created sprints under `/projects/{projectId}/teams/{teamId}/sprints`, and a sprint belonged to a project-team pair. `dev` had the `sprints` table but no routes.
- **Why it did not work:** the team-scoped route depends on `project_teams`, which we do not port (D14). Its routes also depend on `organization_id` and the header (D2).
- **New design:** sprints belong to a project, like work items and tasks, and use the same service class, the same schema style and the same workspace-scoped session as the other planning routes. Dates are validated (`end_date` not before `start_date`), as is status (`planned`, `active`, `completed`).
- **Why it is better:** one pattern for every planning resource, and workspace isolation comes from the existing RLS-aware session.
- **Also fixed:** `app/db/events.py` stored the changed fields as JSON, which failed on dates and UUIDs. Updating a sprint's dates crashed; updating a project's `team_id` or dates would have crashed the same way. The audit log now converts such values to strings, once, in the shared function.
- **Tests:** four new tests (lifecycle and task release on delete, date and status validation, isolation by project and workspace, service-credential check). The full suite passes against a real Postgres: 354 tests, 1 skipped.
- **Revisit if:** sprints need to belong to a team. That comes back together with `project_teams` (D14).

D28. The Alembic baseline is a frozen schema dump, checked against the install script

- **Change:** `alembic.ini`, `migrations/env.py` and `migrations/versions/0001_baseline.py` (plus `0001_baseline.sql`). `./run.sh migrate` runs `alembic upgrade head`. The Dockerfile copies `migrations/` and `alembic.ini`. `alembic` moved to `requirements.txt`.
- **Old design:** two options were open: hand-write about 60 `op.create_table` calls, or have the baseline call `Base.metadata.create_all`.
- **Why they did not work:** a hand-written baseline would miss what the models cannot express (290 RLS policies, 40 triggers, 19 functions, the pgvector extension) and would drift from the real schema. A baseline that calls the live models builds a different database every time the models change, so later migrations would fail on a fresh install.
- **New design:** the baseline runs a frozen `pg_dump` of the schema built by `app.db.install`, with the session `SET` lines removed. Every later change is a normal Alembic migration on top of it. `migrations/env.py` connects as `DATABASE_ADMIN_URL`.
- **Why it is better:** a fresh database from `alembic upgrade head` is identical to the installed one today and stays identical to what it was when the baseline was cut. RLS, triggers and functions come along for free and are not forgotten.
- **Proof:** `tests/test_migrations.py` builds a database by migration and compares it with the installed one (tables and RLS flags, policies, triggers, functions, indexes, constraints), then checks that the models show no

drift. Index predicates and CHECK text are compared by name and kind, because `pg_dump` restores print them differently without changing their meaning.

- **Kept for now:** `app/db/install.py` still exists, and the tests build their database with it. When the team trusts migrations in production, switch the tests to `alembic upgrade head` and retire the install script. Until then the new test fails if the two ever differ.
- **Revisit if:** the production Postgres is a different major version than the one that made the dump (16.2). Re-run the migration test against it.

D29. The CRUD routes accept tokens only; the service key is for the backend

- **Change:** the workspace, member, team, project, work item, task, sprint, comment, doc and channel routes now require a Bearer token. The service key is refused on them with 401. The service key still creates and lists workspaces (`POST /workspaces`, `GET /workspaces`), mints tokens (`POST /auth/tokens`) and runs the existing admin routes.
- **Old design:** these routes (about 80) accepted only the service key, with the workspace taken from the URL. Any caller holding the key could act in any workspace as no one in particular.
- **Why it did not work:** the key carries no identity, no workspace limit and no permissions, so the engine itself could not enforce who may do what. Every rule had to be enforced by whatever called it.
- **New design:** each route checks the token's workspace against the URL (403 on a mismatch), then its scope, then that the member is still active. New scopes: `identity:read/write`, `planning:read/write`, `collab:read/write`, and `admin`. Reads need the read scope; every other method needs the write scope. Members (create, update, delete) and workspace update and delete need `admin`. A token with `admin` passes every scope check except the backend-only routes; atom tokens can never carry it.
- **Why it is better:** a stolen or misused token is limited to one workspace and the scopes it was minted with. Removing a member, or setting them to `suspended`, stops their token on the next request (tested). The permissions live in the engine, not in each caller.
- **Tests:** `tests/test_crud_auth.py` (8 tests): key refused, workspace lock, read cannot write, scopes do not cross areas, admin-only routes, suspended member, admin never on agent tokens, workspace create stays backend-only. Existing CRUD tests now call through a test client that mints an admin token for the workspace in the URL. The full suite passes: 363 tests, 1 skipped.
- **Not done yet (follow-ups):**
- Atoms, approvals, skills, memory, files, meetings and conflicts routes still accept the service key as well as tokens.
- **Revisit if:** a trusted internal service must call CRUD routes without a user. Give it its own token type instead of re-allowing the key.

D30. User-account routes are added and stay backend-only

- **Change:** `POST/GET /users`, `GET/PATCH/DELETE /users/{user_id}`, all behind the service key. Delete is a soft delete. Email is unique ignoring case.
- **Old design:** `dev` had the global `users` table but no routes. The feature branch had `/users` routes behind the service key and the organization header.
- **Why it did not work as is:** a user belongs to no single workspace, so there is no workspace to put in a token and no row-level security rule that fits. A workspace token must not be able to read or edit accounts across workspaces.

- **New design:** the backend, which authenticates people, manages accounts. Tokens never touch the `users` table through the API.
- **Why it is better:** the one table that spans tenants is reachable only by the trusted backend. Workspace tokens stay confined to their workspace.
- **Known gap:** deleting a user does not suspend their memberships. Until it does, suspend the members as well.
- **Tests:** three tests (CRUD and soft delete, unique and validated email, tokens refused). The full suite passes: 366 tests, 1 skipped.
- **Revisit if:** people need to edit their own profile through the engine. That would be a `users:self` rule where the token's `sub` equals the user id.

D31. Authors come from the token, and private channels and teams are hidden

- **Change:** (1) the author of a message or comment is always the token's member; a body that names someone else is refused (403), and so is a token bound to no member. (2) Channels, their messages and teams are filtered by who is looking. (3) Only a channel's owners and administrators can change its members or privacy, and only a message's author or an administrator can edit or delete it. (4) Changing a team's members or its visibility, or deleting a team, needs `admin`.
- **Old design:** these routes were backend-only, so the backend decided who could see what and who a message was from. `author_id` was taken from the request body, and every list returned the whole workspace.
- **Why it did not work:** once D29 let tokens call these routes, any holder of `collab:write` could post as any member, and any holder of `collab:read` could read private channels, direct messages and private teams. An automated security review of the D29 commit found this, and the code confirmed it.
- **New design:** the visibility rule is applied in the service, in the same query that loads the data, so a hidden channel is reported as missing (404) and its existence is not revealed.
- Members see their own channels.
- Direct (`dm`) and agent channels are members-only, even for an admin.
- Private group, team and project channels are visible to their members and to an admin.
- Everything else is open to the workspace.
- A private team is visible to its members and to an admin.
- A channel's creator becomes its `owner`, and `ChannelService`, `MessageService` and `TeamService` now require the viewer, so a route cannot forget to pass one.
- **Why it is better:** the rules cannot be bypassed by a new route that forgets a filter, and they hold even if the backend is wrong about the caller.
- **Tests:** `tests/test_visibility.py` (7 tests): forged and missing authors, edit and delete by author or admin, private channels, direct and agent channels hidden from admin, channel ownership, private teams, team admin rules. The full suite passes: 373 tests, 1 skipped.
- **Known limits:**
- Docs, projects and tasks have no private flag, so they are visible to the whole workspace. A project that belongs to a private team is not hidden.
- Pinning a message counts as editing it, so only the author or an administrator can pin.
- Comments are not hidden by the visibility of what they are attached to.
- **Revisit if:** docs or projects need to be private. That needs a visibility column on them first.

D32. A human's token subject is their user id, and changes record who made them

- **Change:** (1) When a token is minted for a human member, its subject is that member's user id. If the request names no subject, the id is filled in; if it names a different one, minting fails with 422. AI members keep a free subject. (2) Workspace, member, team, project, work item, task, sprint, comment, doc, channel and message writes now record the token's member in the audit log, and creating a project, work item, task or doc sets its `created_by`.
- **Old design:** `sub` was a free-text label (default `"agent"`), and these writes recorded no author (`created_by` and the audit actor were empty).
- **Why it did not work:** D3 requires the token to say who the person is. With a free label, two tokens for the same person could carry different identities, and nothing tied a token's subject to a real account. With no author recorded, there was no answer to "who changed this".
- **New design:** the person's identity is fixed when the token is made, by the one component allowed to make tokens, and every service receives the acting member from the verified token, not from the request.
- **Why it is better:** the user id in `sub` can be trusted by anything that reads the token, and the audit log can be searched by person.
- **Tests:** three more tests (default and mismatched subject, free subject for AI members, attribution across projects, work items, tasks and the workspace). The full suite passes: 377 tests, 1 skipped.
- **Known limits:**
- Planning and collaboration responses do not show `created_by` for projects and work items (tasks do). The value is stored and can be added to the responses later.
- Existing tokens and any rows written before this change have no author.
- Comment and doc updates and deletes are attributed in the audit log, but there is still no "only the author can edit a comment" rule.
- **Revisit if:** the backend needs to mint a token for a person who has no member in the workspace yet. Today such a token has no member to bind and cannot write.

D33. Tests, Postman and README: ported by intent, not by copy

- **Change:** two feature-branch ideas are now tests on `dev`: the full journey (`tests/test_flow.py`) and "another tenant sees every row as missing" (`tests/test_tenant_isolation.py`, covering members, teams, projects, work items, sprints, tasks, docs and channels, and the 401 and 403 cases). Postman is now an OpenAPI import generated from the app (`docs/postman/`, `./run.sh openapi`) plus a small environment file. The README describes migrations, who can call what, and the new docs.
- **Old design:** about 2,400 lines of pytest tests, a 3,400-line hand-made Postman collection and README text, all written for organizations, the `X-Organization-Id` header, camelCase bodies, uppercase enums and routes such as `/organizations/{id}/members` and `/tasks/by-key/...`.
- **Why it did not work:** almost every line names something that no longer exists (D2, D6, D7, D12). Rewriting them one by one would have kept the old shape and tested nothing new. The unit tests in particular check the camelCase aliasing, the old error envelope and the feature's own schemas.
- **New design:** keep what tests behaviour that still matters, and write it against the real routes, real tokens and a real database. Generate the Postman collection from the app's own OpenAPI description so it cannot drift.
- **Why it is better:** the tests exercise the same auth path as production, and the API reference can never disagree with the code.
- **Not ported:** the feature branch's per-resource integration tests, its unit tests for camelCase, error mapping and schema validation, and its hand-made Postman collection. `dev` already has equivalent coverage in

`test_planning_api`, `test_identity_api`, `test_collaboration_api` and the new auth tests.

- **Tests:** the full suite passes: 380 tests, 1 skipped.
- **Capabilities found only on the feature branch:** task by key, task filters, a member's tasks and projects, user lookup by email, and `hasMore` were added (D34). Archiving a project instead of deleting it was not (no decision to port it).

D34. Five feature-branch capabilities added to `dev`'s API

- **Change:** (1) `GET /workspaces/{id}/tasks/by-key/FLWN-12`. (2) Task list filters `assignee_id`, `status`, `sprint_id` and `backlog`. (3) `GET /members/{id}/tasks` and `GET /members/{id}/projects`. (4) `GET /users?email=`. (5) A `has_more` field on every paged list of the workspace, member, team, user, project, work item, task, sprint, comment, doc, channel and message routes.
- **Old design:** `dev` had none of these. The feature branch had them under organization-scoped, camelCase routes (`hasMore`, `/tasks/by-key/...`, `/members/{id}/tasks`).
- **Why it did not work as is:** the routes belonged to the organization model and returned camelCase (D2, D7). The ideas are useful, and a client needs them: a task is referred to by `FLWN-12` in conversation, a board needs filters, and a pager needs to know when to stop.
- **New design:** the same behaviour on `dev`'s workspace-scoped, token-checked routes, in snake_case. A member's projects are the ones they lead or whose team they belong to. A page asks for one row more than it returns, so `has_more` is exact, including when the last page is full. Statuses and keys are validated (422).
- **Why it is better:** all of it runs through the existing token checks and row-level security, so none of it can read another workspace (tested for by-key and for foreign members). No new tables or columns.
- **Tests:** five tests in `tests/test_planning_queries.py`. The full suite passes: 385 tests, 1 skipped.
- **Known limits:** `has_more` is not on the memory, files, meetings, atoms and conflicts lists, which have their own paging. "My projects" ignores private-team visibility for projects, because projects have no private flag (D31).
- **Revisit if:** projects need archiving, or a board needs sorting and free-text search.

3. Merge order

1. Create `merge/feature-into-dev` from `dev`.
2. Apply D2 to D9 on that branch, each as its own commit.
3. Run the full test suite against Postgres with pgvector, using a role that does not bypass RLS.
4. Open a pull request from `merge/feature-into-dev` into `dev`.
5. Merge `dev` into `main` (fast-forward is possible because `main` has nothing `dev` lacks).

4. Template for new entries

```
### Dn. Short title

- **Change:** what was added, removed or replaced.
- **Old design:** how it worked before, and on which branch.
- **Why it did not work:** the concrete problem.
- **New design:** what replaces it.
```

```
- **Why it is better:** the benefit, and any cost.
- **Revisit if:** the condition that would reopen this decision.
```

5. Producing the PDF

```
pandoc docs/MERGE_DECISIONS.md -o docs/MERGE_DECISIONS.pdf --pdf-engine=xelatex
```